Кам'янець-Подільський національний університет імені Івана Огієнка

Кафедра комп'ютерних наук

ЗВІТ
лабораторної роботи №2
з дисципліни «Об’єктно-орієнтоване програмування»

Виконувала: студентка групи KN1-B25

Сурженко Анастасія

ТЕМА. Інкапсуляція та керування станом об'єктів у C#

Мета: набути практичних навичок реалізації принципу інкапсуляції в об'єктно-орієнтованих програмах мовою C#, навчитися проєктувати класи з контрольованим доступом до їхнього внутрішнього стану, використовувати властивості, модифікатори доступу та сучасні можливості C# для ініціалізації й валідації даних. Сформувати вміння забезпечувати цілісність стану об'єктів, реалізовувати перевірку коректності даних під час їх створення та зміни, а також правильно організовувати взаємодію між об'єктами відповідно до принципів об'єктно-орієнтованого програмування.

Варіант №3

Хід роботи

1. Інкапсуляція даних.

- У класах Car та Customer внутрішні поля зроблено закритими (private).
- Доступ до даних організовано через публічні властивості з відповідними get та set геттерами/сеттерами.

2. Валідація даних.

- У класі Car реалізовано перевірку року випуску автомобіля (рік не може бути меншим за 1886 та більшим за поточний рік). При спробі встановити некоректний рік генерується виняток ArgumentOutOfRangeException.
- Для текстових властивостей (Brand, Model, Name) додано перевірку на порожні значення.

3. Керування станом.

- До класу Car додано властивість IsUnderRepair з приватним сеттером private set.
- Для безпечної зміни стану додано методи StartRepair() та FinishRepair(), що унеможливлює некоректну пряму зміну стану з зовнішнього коду.

Посилання на GitHub-репозиторій:

https://github.com/anasstasiiykaa/oop-2026-Surzhenko/tree/main

Лістинги

Файл Models/Car.cs

```
namespace AutoServiceApp.Models;

public class Car
{
    private string _brand = string.Empty;
    private string _model = string.Empty;
    private int _year;

    public string Brand
    {
        get => _brand;
        set => _brand = string.IsNullOrWhiteSpace(value)
            ? throw new ArgumentException("Марка автомобіля не може бути порожньою.", nameof(value))
            : value.Trim();
    }

    public string Model
    {
        get => _model;
        set => _model = string.IsNullOrWhiteSpace(value)
            ? throw new ArgumentException("Модель автомобіля не може бути порожньою.", nameof(value))
            : value.Trim();
    }

    public int Year
    {
```

```
            get => _year;
            set
            {
                int currentYear = DateTime.Now.Year;
                if (value < 1886 || value > currentYear)
                {
                    throw new ArgumentOutOfRangeException(
                        nameof(value),
                        $"Рік випуску ({value}) некоректний. Очікується діапазон від 1886 до {currentYear}."
                    );
                }

                _year = value;
            }
        }

        public bool IsUnderRepair { get; private set; }

        public Car(string brand, string model, int year)
        {
            Brand = brand;
            Model = model;
            Year = year;
            IsUnderRepair = false;
        }

        public void StartRepair()
        {
            if (IsUnderRepair)
```

```
            {
                Console.WriteLine($"Автомобіль {Brand} {Model} вже знаходиться на ремонті.");
            }
            else
            {
                IsUnderRepair = true;
                Console.WriteLine($"Автомобіль {Brand} {Model} успішно прийнято на ремонт.");
            }
        }

        public void FinishRepair()
        {
            if (!IsUnderRepair)
            {
                Console.WriteLine($"Автомобіль {Brand} {Model} не перебуває на ремонті.");
            }
            else
            {
                IsUnderRepair = false;
                Console.WriteLine($"Ремонт автомобіля {Brand} {Model} завершено.");
            }
        }
    }
```

Файл Models/Customer.cs

```
namespace AutoServiceApp.Models;
```

```
public class Customer
{
    private string _name = string.Empty;

    public string Name
    {
        get => _name;
        set => _name = string.IsNullOrWhiteSpace(value)
            ? throw new ArgumentException("Ім'я клієнта не може бути порожнім.", nameof(value))
            : value.Trim();
    }

    public Customer(string name)
    {
        Name = name;
    }
}
```

Файл Program.cs

```
using AutoServiceApp.Models;

namespace AutoServiceApp;

class Program
{
    static void Main(string[] args)
    {
```

```
Console.WriteLine("=== Лабораторна робота №2: Інкапсуляція та Валідація ===\n");

// 1. Створення коректних об'єктів
var car1 = new Car("BMW", "X5", 2020);
var customer1 = new Customer("Олександр Коваленко");

Console.WriteLine($"Власник: {customer1.Name}");
Console.WriteLine($"Автомобіль: {car1.Brand} {car1.Model}, {car1.Year} рік");

// 2. Демонстрація керування станом об'єкта
Console.WriteLine("\n--- Перевірка стану ремонту ---");
Console.WriteLine($"Початковий стан (в ремонті?): {car1.IsUnderRepair}");

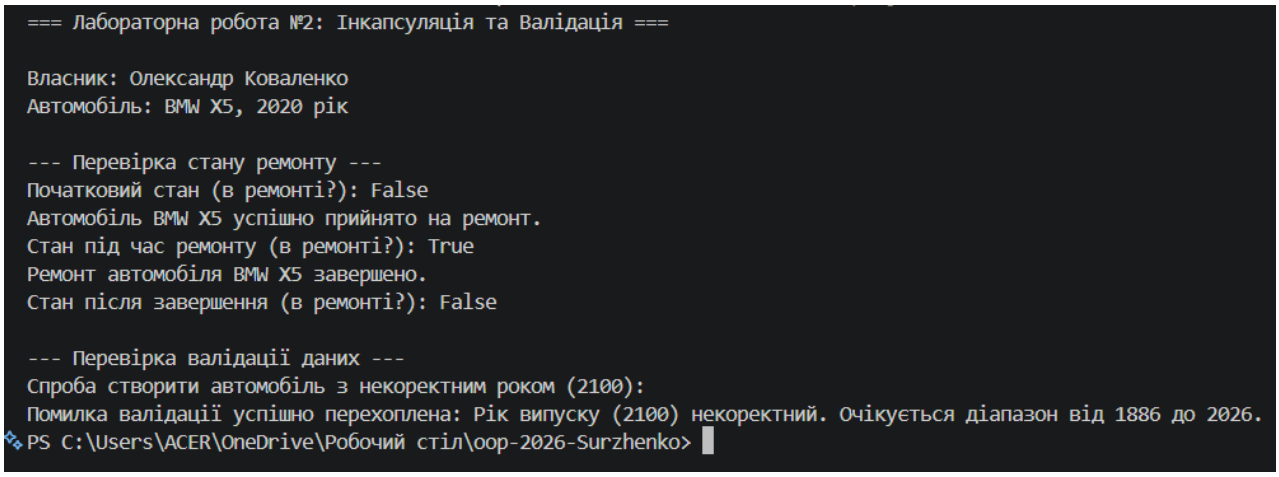
car1.StartRepair();
Console.WriteLine($"Стан під час ремонту (в ремонті?): {car1.IsUnderRepair}");

car1.FinishRepair();
Console.WriteLine($"Стан після завершення (в ремонті?): {car1.IsUnderRepair}");

// 3. Демонстрація обробки некоректних даних (валідація)
Console.WriteLine("\n--- Перевірка валідації даних ---");
try
{
    Console.WriteLine("Спроба створити автомобіль з некоректним роком (2100):");
```

```
            var invalidCar = new Car("Audi", "A6", 2100);
        }
        catch (ArgumentOutOfRangeException ex)
        {
            Console.WriteLine($"Помилка валідації успішно перехоплена: {ex.Message}");
        }
    }
}
```

Скріншот результату після запуску програми:



Висновок: у ході виконання лабораторної роботи було опановано принцип інкапсуляції мовою C#. Завдяки використанню приватних полів та публічних властивостей з валідацією забезпечено цілісність даних об'єктів. Застосування методів для зміни стану IsUnderRepair з обмеженням прямого доступу дозволило унеможливити перехід об'єктів у некоректні стани, що підвищує надійність та безпеку програмного коду.